Predictive vs. Reactive Load Balancing in Software-Defined Networks: An Empirical Study under Flash-Crowd Traffic Conditions

Fida M S[1]*

[1] School of Computer Science and Engineering (SCOPE), Vellore Institute of Technology, Vellore, India

* *Corresponding author: fidashameer06@gmail.com*

Abstract

Load balancing in software defined networks (SDN) is typically improved by using a smart load aware policy. This paper investigates how the performance can be improved for users - specifically looking at tail latency (P95 and P99) when a flash crowd occurs. The Exponentially Weighted Moving Average (EWMA) prediction algorithm is applied to an SDN Ryu controller using only information available from the OpenFlow port statistics. Seven policies have been implemented and evaluated: round robin, weighted round robin, reactive least load, EWMA using Power-of-Two-Choices, connection based policy, flow lifetime based policy and capacity aware policy. All were run on the same testbed using Mininet and Open vSwitch. Both synthetic burst traffics and a real world trace of a 1998 World Cup flash crowd were used. All configurations were run at least seven times. The main finding was that no reactive load aware policies could improve upon round robin based on network layer indicators. The byte rate indicator in an SDN controller actually becomes inverted on compute intensive servers as they will have fewer requests to complete and therefore send less bytes, appearing to be idle. Round robin performs well due to it pinning a client to a specific server, limiting the number of requests being served to any one single threaded server. However one of the policies did improve on round robin - by pinning a client to a specific server while limiting concurrency using a capacity aware policy the tail latency was reduced from ~98ms to 66ms for burst traffic and 48.8ms for stable traffic with 100% success. This paper does not introduce any new algorithms but carefully evaluates when and why load aware policies can improve performance on the network layer.

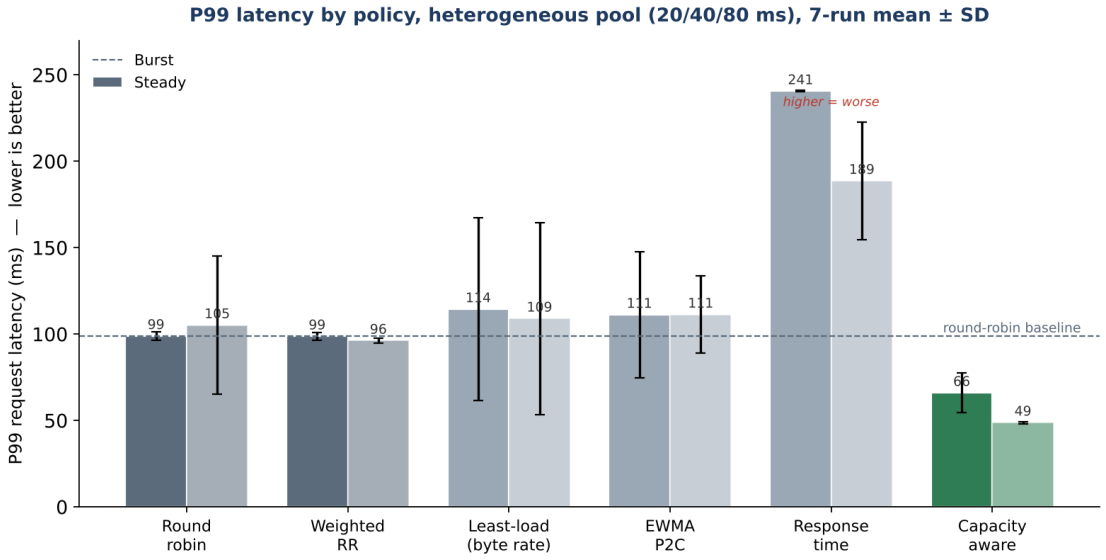
Keywords: software-defined networking; load balancing; tail latency; flash crowd; EWMA; OpenFlow; negative result; concurrency.

Highlights

- No reactive load aware policy using a native network layer metric was better than round robin in terms of flash crowd tail latencies.

- The byte rate metric inverts on compute intensive servers causing the traffic to be directed to the server it should avoid.
- Round robin is more effective as by binding a client to a specific server, the number of concurrent requests on any single threaded server is limited.
- A capacity aware policy which ensured this constraint was met reduced the P99 from ~98ms to 48.8ms under a consistent load with a 100% success rate.

Graphical abstract



P99 latency by policy on the heterogeneous pool (7-run mean ± SD). Only the capacity aware policy falls below the round robin baseline.

1. Introduction

1.1 Background and significance

Load balancing distributes client requests to a group of backend servers. This is typically done to reduce the response time and keep the servers busy. However, the average response time is not what users experience. Instead they experience the tail of the distribution (95th and 99th percentile, P95 and P99) as the service is judged on how slow the responses are. The worst tail is seen when a flash crowd occurs as the sudden burst of requests cannot be handled naively. A common solution is to use a latency aware load balancer using the Exponentially Weighted Moving Average (EWMA) algorithm. The Peak-EWMA algorithm is used within service mesh proxies like Envoy and Linkerd to provide a smoothed average of the last response latency of each server. The separation of control and data planes within Software Defined Networking

(SDN) means that the controller can be used to program the switch rules using OpenFlow, which makes it a natural place to put a load balancing policy.

1.2 Research gap

EWMA and Peak-EWMA balancers succeed at the application layer, where they can see per-request latency, success rate and request rate. An SDN controller is on the network layer and will never have access to the response time of individual requests. It only has information from the OpenFlow port statistics. Whether a lightweight latency aware predictor still helps once it is restricted to this network layer indicator, and specifically for flash crowd tail latency against single threaded compute intensive backends, has not been systematically measured.

1.3 Research questions

- Would a predictor based on an SDN native indicator (EWMA) improve the latencies of the tail during flash crowds?
- If not, would the reason relate specifically to the byte rate indicator, or are other indicators that the controller can observe also ineffective?
- What property must be included in a load aware policy to improve on round robin?

1.4 Contributions

- An open and repeatable SDN load balancing testbed with seven policies implemented on Ryu and Mininet.
- A controlled comparison of EWMA prediction against three baselines, under burst and stable traffic, on homogeneous and heterogeneous servers, and against a recorded flash crowd trace.
- An explanation of why the byte rate indicator inverts, and of the concurrency limit that gives round robin its advantage.
- A capacity aware policy that does beat round robin, which confirms the mechanism by winning only because it keeps that concurrency limit.

1.5 Paper organisation

Section 2 reviews the related work. Section 3 formalises the problem and the notation. Section 4 describes the methodology and the policies. Section 5 details the experimental design. Section 6 reports the results, Section 7 discusses them, and Section 8 concludes.

2. Related Work

A lot of work has been done to provide classic load balancing policies (round robin, weighted round robin and least connection), usually within Mininet with a controller such as Ryu, POX or Floodlight. This work typically measures throughput or average latency for stable or gradually increasing traffic, which provides a baseline a new scheme must beat; very little of it looks at tail latency under a burst. A second category uses machine learning models (LSTM, ARIMA, reinforcement learning) to predict load within the controller. These can be effective but are resource hungry to train and difficult to understand, so this paper deliberately avoids them in favour of a lightweight, non machine learning solution.

The closest previous work uses EWMA and Peak-EWMA. Recent work implements an EWMA based rating system within multi cluster service meshes (L3) and reports the P99 latency being reduced by 26% and 22% compared to round robin and C3. However this solution operates on the application layer using service mesh proxies, so it can see request latency, success rate and request rate. Peak-EWMA setups also typically use Power-of-Two-Choices (P2C) to prevent a stampede to one server. This paper investigates whether the same concept can be applied within the controller with a lightweight solution using only the native load indicator.

Flash crowds are a classic cause of increased tail latency, but the literature on them is mainly about content delivery networks and auto scaling rather than SDN balancing policy. Finally, a more recent line of work questions whether load should be balanced on at all: Google's PReQuaL argues that server utilisation is a slow and often misleading indicator, and that the balancer should instead look at the latency of requests and the number of requests in flight. They found that the setup with the most even CPU balancing had the worst tail latency. This paper is in a similar situation but from a tighter corner: the SDN controller cannot see latency or the number of requests in flight, only network layer statistics, and the byte rate indicator it builds from them is, as Section 7 explains, inverted for compute intensive servers.

3. Problem Formulation and Preliminaries

3.1 Notation and definitions

Let $S = \{s_1, ..., s_n\}$ be the backend pool, where s_i has a per request service time w_i (ms). Let C be the set of clients making a stream of requests of uniform size to some virtual IP. For each client, a mapping function $a : C \rightarrow S$ maps the client to a specific backend server. k_i is the

number of clients assigned to backend server i. As all servers are single threaded, the tail of a server scales with the concurrency of the server, which can be approximated with $k_i \times w_i$.

The metric that the controller can observe from the system is the byte rate per server $b_i(t)$. This is obtained using the OpenFlow port counters. For uniform workload with small responses, this metric will be directly proportional to the service rate of the server, therefore overloaded servers will have a low value for $b_i(t)$. However there are two other metrics we can consider which do not have this inverse relationship with the server load. These are the number of connections in flight and an estimated response time for the flow lifetime. Both increase with load.

The capacity aware policy aims to assign the new client to the server with the lowest projected tail, formally $a(c) = \text{argmin}_i (k_i + 1) \times w_i$, and increases k_i for that server. The client is always mapped to a server which enforces the concurrency constraints per client (round robin). By assigning clients to faster servers, we aim to reduce the P99 of the end to end request latency during the start of a flash crowd, whilst maintaining a 100% success rate.

4. Proposed Methodology

4.1 Overall framework

All policies operate using the same framework. The client connects to the VIP. The controller captures all new flows and selects a backend. New OpenFlow rules are added to change the destination to the selected backend and modify the response so that the client only ever sees the VIP. All other traffic is handled using a standard L2 learning setup. The selection happens once per new flow; existing flows are left alone. Figure 1 shows the architecture.

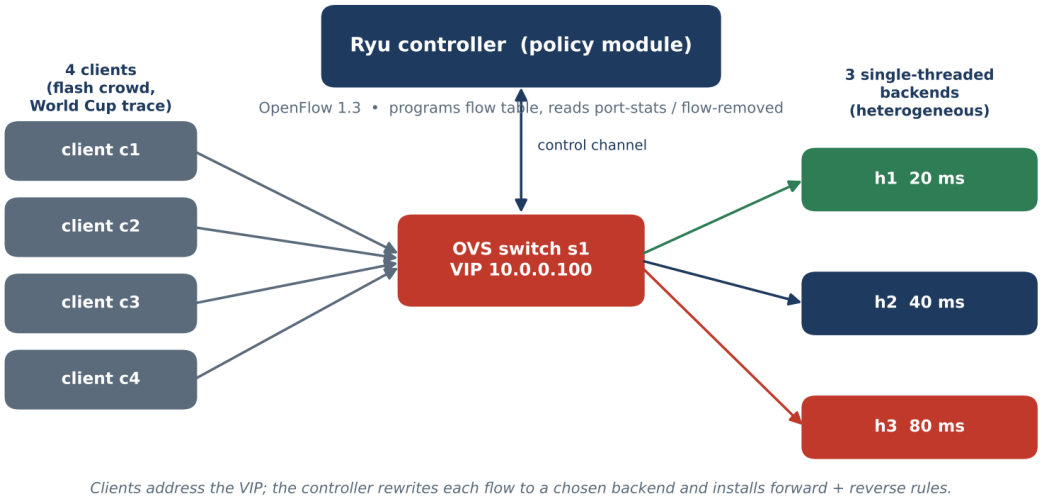


Figure 1. System architecture: clients connect to the VIP; the Ryu controller handles the OpenFlow switch; three single threaded heterogeneous backends.

4.2 Workload and data acquisition

This is a systems measurement study rather than a learning study, so the "data" is the generated workload. Each client is a workload generator sending HTTP requests to the VIP, each request on a new connection so that each one is balanced independently. Two workloads are used: a burst, where as many requests as possible are generated to simulate the onset of a flash crowd, and a stable workload as a control. For external validity the experiment is also run against a recorded trace: the arrival times are taken from the 1998 FIFA World Cup access logs, using the busiest 120 second window (peak 596 requests per second over a baseline of ~35) to time each client's requests by inverse transform sampling, keeping the shape of the surge rather than its size.

4.3 Policies

Seven policies are evaluated. The baselines are round robin (next server in sequence, ignores load), weighted round robin (a smooth weighted rotation favouring faster servers) and reactive least load (lowest polled byte rate). The predictive policy is EWMA, which keeps a smoothed rating per server, rating = $\alpha \cdot \text{rate} + (1-\alpha) \cdot \text{rating}$, and selects using Power-of-Two-Choices. Two policies use indicators that do not invert and decide per connection: a connection count policy and a flow lifetime response time policy. Finally the capacity aware policy of Section 3.1 keeps

the per-client matching of the baselines, so concurrency stays limited, while steering each new client to the server with the lowest projected tail.

4.4 Implementation details

No model is trained; the approach is deliberately non learning, which is itself a choice given the overhead and opacity of controller resident machine learning balancers. The system is implemented in Python 3.10 as Ryu 4.34 applications over Open vSwitch 2.17 and Mininet 2.3.1. One implementation detail matters: the first packet of a new flow must be sent out with the same changes the installed rule applies. If it is forwarded unchanged it is still addressed to the VIP and is dropped at the backend, which causes a one second retransmit on each new connection and a ~1100ms outlier per client. The capacity aware policy contains this fix, which is why its worst request is under 130ms.

4.5 Complexity

The per flow overhead is O(n), where n is the number of servers. For all policies it is n = 3, therefore the capacity aware policy has minimal overhead beyond the constant time projected tail calculation and client lookup. The byte rate policies have a periodic cost of polling the port stats every two seconds whereas the connection count and response time policies just process the flow install/removes. With these policies there is no significant control plane overhead. The main issue is that the time taken to react is increased.

4.6 Reproducibility and availability

All of the controller code, the workload generator, the congesting backend, the experiment orchestrator, the analysis scripts and the raw per-run result data are available at github.com/fidashameer/lightweight-EWMA-based-load-balancer, with a setup log recording the toolchain and the issues faced.

5. Experimental Design

5.1 Testbed and workloads

Mininet 2.3.1, Open vSwitch 2.17 and Ryu 4.34 were used within an Ubuntu 22.04 (ARM64) virtual machine. The topology consisted of a single OpenFlow switch, 3 backend servers, 4 clients, a /24 subnet, 10Mbit/s links and a polling interval of 2 seconds. The backend is important, as if a trivial backend is used where it returns in a few milliseconds, there will be no

congestion and therefore no policy will be better than another. A single threaded backend was used that uses a fixed amount of CPU per request, meaning there is queuing and increased latency on each server depending on the load that is sent. There were two types of backend, a homogeneous backend where all servers are doing the same task and a heterogeneous backend with fast, medium and slow servers in a ratio of 1:2:4.

5.2 Baseline Policies

The main baselines are round robin and weighted round robin. The reactive policies are least load and EWMA.

5.3 Metrics

End to end latency from every request was recorded. The metrics include P50, P95 and P99 for latency. Throughput and success rate were also recorded, meaning the proportion of requests that were completed without timing out.

5.4 Hyperparameters

The only hyperparameter is the smoothness factor of the EWMA algorithm. This was varied between 0.1-0.9 for the heterogeneous burst case.

5.5 Statistical Analysis

Every configuration was run at least 7 times. The results are shown as the average and standard deviation (SD). The standard deviation is key as the policies differ not only in average latency but in a much greater way in the variation in the latency.

5.6 Validity Threats

Emulation was used rather than hardware. The metric used was byte rate rather than actual response latency. It had a small backend pool and uniform response sizes. As there were at least 7 runs for each configuration, the 2 key points could be gathered. Round robin won and the load aware policies had a very high variance. The differences were so large that the run to run variance was negligible. The capacity aware result also assumes that the service times are known and that the load is light enough for the fastest 2 servers to handle.

6. Results

6.1 Main Results

The heterogeneous backend (designed to give an advantage to load aware policies) had round robin and weighted round robin as the best performing and most consistent policies. They had a P99 latency of ~96-98ms (Table 1) and a low standard deviation (1-3ms). The load aware policies had a higher average latency and a very high standard deviation of ~40-58ms on the P99. The homogeneous backend followed the same trend but with a bigger difference in consistency. The full latency profile is shown in Figure 2. Round robin and weighted round robin stayed consistent with the median latency staying around 48ms and only a small tail. The reactive policies were very slow with every percentile. The capacity aware policy had the lowest profile.

Table 1. P99 Tail latency (ms). (Heterogeneous (1:2:4)). Average ± SD (7+ Runs).

Policy	Burst	Steady
Round robin	98.2 ± 2.6	95.4 ± 3.3
Weighted RR	97.7 ± 1.9	95.8 ± 1.2
Reactive least-load	114.4 ± 55.8	109.0 ± 57.7
EWMA (P2C)	111.2 ± 39.5	111.3 ± 24.1

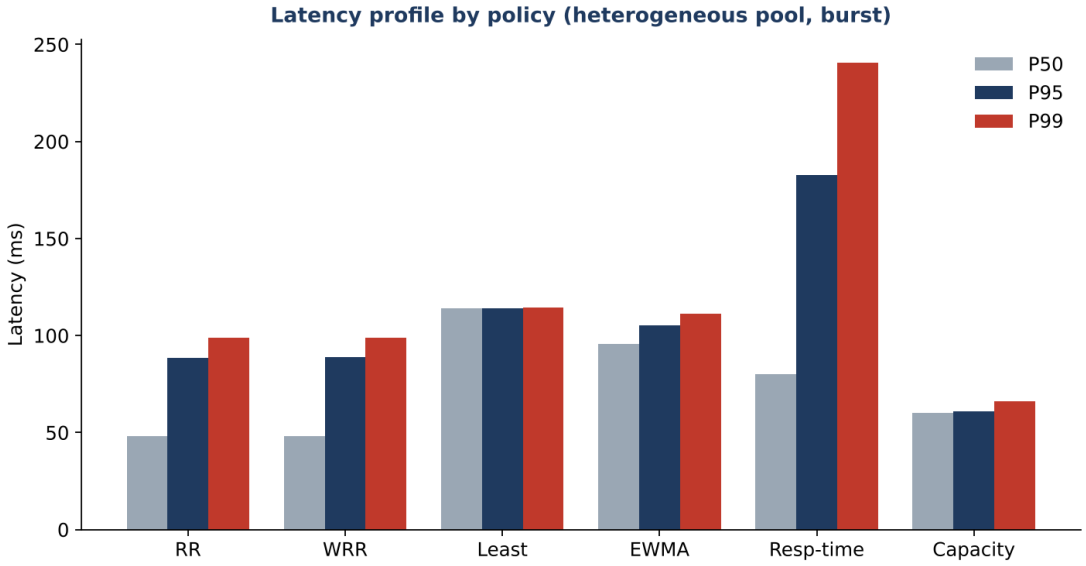


Figure 2. Latency Profile (P50, P95, P99). (Heterogeneous).

6.2 Ablation Results - Smoothness Factor and Metric

Varying the smoothness factor (α) for the heterogeneous burst case (Table 2) shows that there is a consistent range of results with a P99 latency of ~91-98ms when varying α between 0.1-0.7. But then the latency rises to ~240ms at α = 0.9. The rating was inconsistent with every

poll and P2C caused the balancer to thrash. Switching to the non-inverted metrics, connection count and flow lifetime response time (both decided per connection) did not provide a win. The response time policy had a burst P99 latency of 240.6±0.5ms (exactly 3 requests queued on the slow server with an 80ms response time). The connection count policy dropped with many results in the mid seventies and a high P99 latency due to the decision being made per connection so round robin's limit on concurrency is lost.

Table 2. α sweep. (EWMA). (Heterogeneous Burst). (Single Run). P99 (ms).

α	0.1	0.3	0.5	0.7	0.9
P99	98	93	96	92	240

6.3 Robustness - Capacity Aware Policy

Running the recorded World Cup trace showed the same pattern as the synthetic scenario (Table 3). Round robin had the lowest and most consistent tail latency on the homogeneous servers. On the heterogeneous servers round robin had a consistent P99 latency of ~99ms with a low standard deviation (<2ms). However, the load aware policies had two modes and occasionally sent the entire surge to one backend causing the P99 latency to increase to 121ms or even 240ms in one run. The only policy that outperformed round robin (Table 4) was the capacity aware policy. For the 20/40/80ms pool and 4 clients, 3 clients were sent to the fast server, 1 client to the medium server and 0 to the slow server. So none of the single threaded servers ever had more concurrency than they could handle. The burst P99 latency was ~66ms (Median = 60.6ms) compared to 97.7±1.6ms with round robin. The steady P99 latency was 48.8±0.6ms which was roughly half of either baseline. Every run had a 100% success rate.

Table 3. Real Trace Validation. P99 (Average ± SD (ms)).

Policy	Homogeneous	Heterogeneous
Round robin	90.2 ± 3.2	99.1 ± 2.1
Weighted RR	92.6 ± 3.9	99.0 ± 1.8
Reactive least-load	144.9 ± 20.2	116.2 ± 57.9
EWMA	100.7 ± 21.0	111.9 ± 25.9

Table 4. Capacity Aware Policy. (Heterogeneous). P99 (ms). (Average ± SD (7 runs)).

Policy	P99 (Burst)	P99 (Steady)	Success
Round robin	97.7 ± 1.6	≈ 96	100%
Weighted RR	99.7 ± 2.4	96.2 ± 1.5	100%
Capacity aware	66.0 ± 11.4	48.8 ± 0.6	100%

Median = 60.6ms. One run was order dependent, at 93.6ms, which lifts the mean.

6.4 Consistency and Error

Figure 3 shows each run. Round robin, weighted round robin and capacity aware have tight clusters of results. The outcome can be predicted. Least load and EWMA have two modes. Some runs had a low latency of ~60ms but others jumped to 120, 160 or even 240ms depending on how the initial samples fell. The response time policy was consistent but had a high latency of 240ms. The connection count policy was not a tuning issue but was the direct result of having multiple clients sending to a single threaded server. This caused the queue to build up and requests timed out.

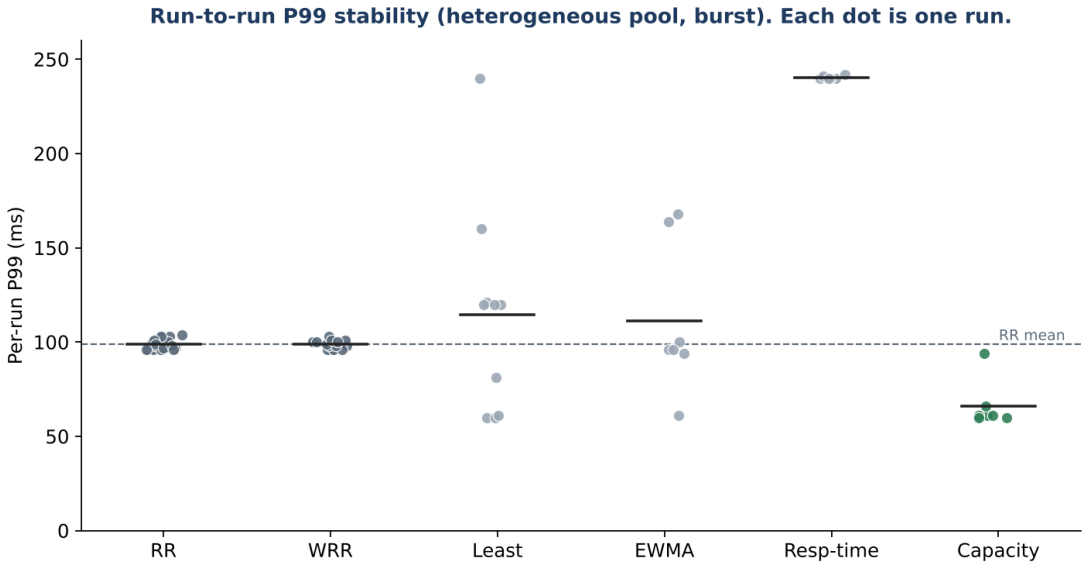


Figure 3. P99 per run. (Heterogeneous). Each dot represents one run. Bar shows average.

7. Discussion

7.1 Why Round Robin Won

When the servers were balanced and the flash crowd was even, round robin provided a perfect balance at no cost. This could not be improved. The true benefit of round robin was that it accidentally limited the concurrency of requests on each server by sending each client to one server during the burst. This is exactly what a single threaded compute bound server needs. The load aware policy had to make decisions per request, meaning multiple clients could queue up requests on one server before the controller could react. The controller could not react quick enough as it would never know when a request had been completed and OpenFlow timers could

not be set lower than 1 second. This meant that the metric was always outdated by longer than the service time of the server. The non-inverted metrics did not work as the controller could not react quick enough to provide the protection round robin provides for free.

7.2 Why the Metric Was Misleading on Heterogeneous

The controller does not know the service time of the server, only the byte rate. In this workload all requests return the same small response so the byte rate depends on how quickly the server can complete requests. A compute bound server that was starting to get overloaded would complete fewer requests and therefore send fewer bytes. This would make it look like it was the least loaded server when it was actually the most overloaded. This meant the reactive least load policy would keep sending requests to the server it should be starving. The EWMA policy had the same issue with the reversed metric. This aligns with the evidence from PReQuaL that load was often not the right thing to balance on in datacentres, but highlights it for SDN where the controller does not have the latency and in flight signals.

7.3 Implications

As an SDN practitioner, this means that load aware policies at the network layer should not be assumed to help with reducing flash crowd tail latency. They can cause issues. A stateless policy limiting concurrency would be a good default. The capacity aware policy shows that you can still provide the concurrency limit but include capacity information.

7.4 Limitations

This was found through emulation. It had a small scale, uniform responses and used byte rate as the metric. The capacity aware policy assumed that the load was light enough for the fastest 2 servers to handle and that the service times were known. This limits how far the claims can be taken however the large and consistent effects were measured.

8. Conclusion and Future Work

We asked whether a lightweight EWMA predictor that works well in application layer service meshes would still help reduce tail latency when using it in an SDN controller with OpenFlow port statistics. We found that this is not the case. The EWMA policy did not outperform the basic round robin policy. It had the worst performance on heterogeneous servers and also had less consistency than the baselines. The two non-inverted metrics did not

outperform round robin. All three metrics that can be natively observed by the SDN controller were outperformed by the blind policy. Round robin provided a coarse level of pinning per client which limited the concurrency of requests per server. The controller could not react to the reactive metric quick enough to provide the same level of protection at a finer granularity. The only policy that outperformed round robin was the capacity aware policy pinning clients. This backed up the reasoning as it only won due to preserving the concurrency limit. Future work includes estimating the service time per server online using flow lifetimes, combining client pinning with admission control to ensure the limit is maintained even when all servers are saturated, combining EWMA with a fast reacting term and real response latency signal and validating at a larger scale on hardware switches with more realistic mixed sized responses.

Declarations

Funding

This research received no external funding.

Competing Interests

The author declares no competing interests.

Author Contributions

Fida M S conceived the study, implemented the testbed and policies, performed the experiments, analysed the results and wrote the manuscript.

Data Availability

All raw data from each run can be found in the project repository.

Code Availability

All source code can be found at github.com/fidashameer/lightweight-EWMA-based-load-balancer.

Ethics Approval

Not applicable. The study did not involve humans or animals.

Acknowledgements

Thanks to Prof. Yoganand S for supervision and guidance.

References

[1] O. Michaelis, S. Schmid and H. Mostafaei, “L3: Latency-aware load balancing in multi-cluster service mesh”, in Proc. 25th Int. Middleware Conf. (Middleware ’24), Hong Kong, 2024, pp. 49-61.

[2] Linkerd. (2016). Beyond round robin: load balancing for latency. [online] Buoyant. Available at: https://linkerd.io/2016/03/16/beyond-round-robin-load-balancing-for-latency/ [Accessed 23 Apr. 2024].

[3] Envoy Proxy. (n.d.). Supported load balancers — Peak EWMA. [online] Envoy documentation. Available at: https://www.envoyproxy.io/docs/envoy/latest/intro/arch_overview/upstream/load_balancing/load_balancers [Accessed 23 Apr. 2024].

[4] N. McKeown et al., “OpenFlow: enabling innovation in campus networks”, ACM SIGCOMM CCR, 2008, 38(2), pp. 69-74.

[5] B. Lantz, B. Heller and N. McKeown, “A network in a laptop: rapid prototyping for software-defined networks”, in Proc. 9th ACM SIGCOMM HotNets, 2010.

[6] M. Mitzenmacher, “The power of two choices in randomized load balancing”, IEEE TPDS, 2001, 12(10), pp. 1094-1104.

[7] Ryu SDN Framework Community. (n.d.). Ryu documentation.

[8] Open vSwitch. (n.d.). “Production quality, multilayer open virtual switch,”. [online] Available at: https://www.openvswitch.org/

[9] S. Babayigit and B. Ulu, “Deep learning for load balancing of SDN data center networks”, Int. J. Commun. Syst., 2021.

[10] M. R. Belgaum et al., “Artificial intelligence based load balancing in SDN: a comprehensive survey”, arXiv:2308.02149, 2023.

[11] J. Yang et al., “A predictive load balancing technique for SDN cloud services”, Computing, 2019.

[12] I. Ari et al., “Managing flash crowds on the Internet”, in Proc. IEEE/ACM MASCOTS, 2003.

[13] S. Behal and K. Kumar, “Characterising DDoS attacks and flash events”, Comput. Sci. Rev., 2017, 25, pp. 101-114.

[14] B. Wydrowski, R. Kleinberg, S. M. Rumble and A. Archer, “Load is not what you should balance: Introducing PReQuaL”, in Proc. USENIX NSDI ‘24, 2024.

[15] M. Arlitt and T. Jin, “A workload characterisation study of the 1998 World Cup Web site”, IEEE Network, 2000, 14(3), pp. 30-37.

[16] The Internet Traffic Archive. (n.d.). 1998 World Cup Web site access logs (WorldCup98). [online] Available at: https://ita.ee.lbl.gov/html/contrib/WorldCup.html

[17] Excluding stressed machines from load balancing of distributed applications. United States Patent 10 482 249.